Cloud Security Capstone Project

Secure Three-Tier Application on AWS using Terraform

Name: Supriya Gawali

PRN: 2124UCSF1019

1. Introduction and Objective

This project deploys a secure, scalable-by-design three-tier web application on AWS entirely with Terraform, and demonstrates its security controls with evidence from the live deployment.

The objective was to follow AWS security best practices: a web tier (Application Load Balancer) in public subnets, an application tier (EC2) in a private subnet, and a database tier (Amazon RDS) in an isolated private subnet that accepts connections only from the application server. Network flow is logged with VPC Flow Logs and account activity is audited with AWS CloudTrail.

Item	Value
Cloud provider	Amazon Web Services
Region	Asia Pacific (Sydney), ap-southeast-2
Infrastructure as Code	Terraform 1.16 with AWS provider 5.x
Resources created	50
Application	Apache web server on Amazon Linux 2023
Database	Amazon RDS for MySQL 8.0 (db.t3.micro)

2. Architecture Overview

Traffic enters through the Internet Gateway to an Application Load Balancer in the public subnets. The ALB forwards HTTP requests to the EC2 application server in a private application subnet, and the application server alone can reach the RDS database in a private database subnet. Everything spans two availability zones.

Tier	Subnets	Resources	Internet route
Web	Public: 10.0.1.0/24, 10.0.2.0/24	Application Load Balancer, NAT Gateway	Yes, through the Internet Gateway
Application	Private: 10.0.11.0/24, 10.0.12.0/24	EC2 server (no public IP)	Outbound only, through the NAT Gateway
Database	Private: 10.0.21.0/24, 10.0.22.0/24	RDS MySQL	None

The security groups form a chain: the Internet may reach only the ALB on port 80, the ALB may reach only the application server, and the application server may reach only the database on port 3306.

3. Terraform Implementation

The infrastructure is split into nine Terraform files so each tier and each security control can be reviewed on its own.

File	What it builds
versions.tf	Terraform and AWS provider versions, default tags
variables.tf	Region, availability zones, CIDR ranges, instance sizes
network.tf	VPC, six subnets, Internet Gateway, NAT Gateway, three route tables
security_groups.tf	ALB, application and database security groups with SG-to-SG rules
alb.tf	Application Load Balancer, target group, HTTP listener
compute.tf	EC2 application server, IAM role for Session Manager
database.tf	RDS MySQL instance and DB subnet group
monitoring.tf	VPC Flow Logs to CloudWatch, CloudTrail to a private S3 bucket
outputs.tf	ALB URL, instance ID, RDS endpoint, log group and bucket names

The deployment followed the standard workflow: `terraform init`, `terraform validate`, `terraform plan -out tfplan`, then `terraform apply`. The plan reported 50 resources to add. The security-critical lines in the code are:

```
# database.tf
publicly_accessible         = false
storage_encrypted           = true
```

4. Deployment Evidence

The screenshots below were taken from the AWS Console (region Asia Pacific (Sydney)) and the terminal after `terraform apply` completed.

4.1 Terraform apply and application

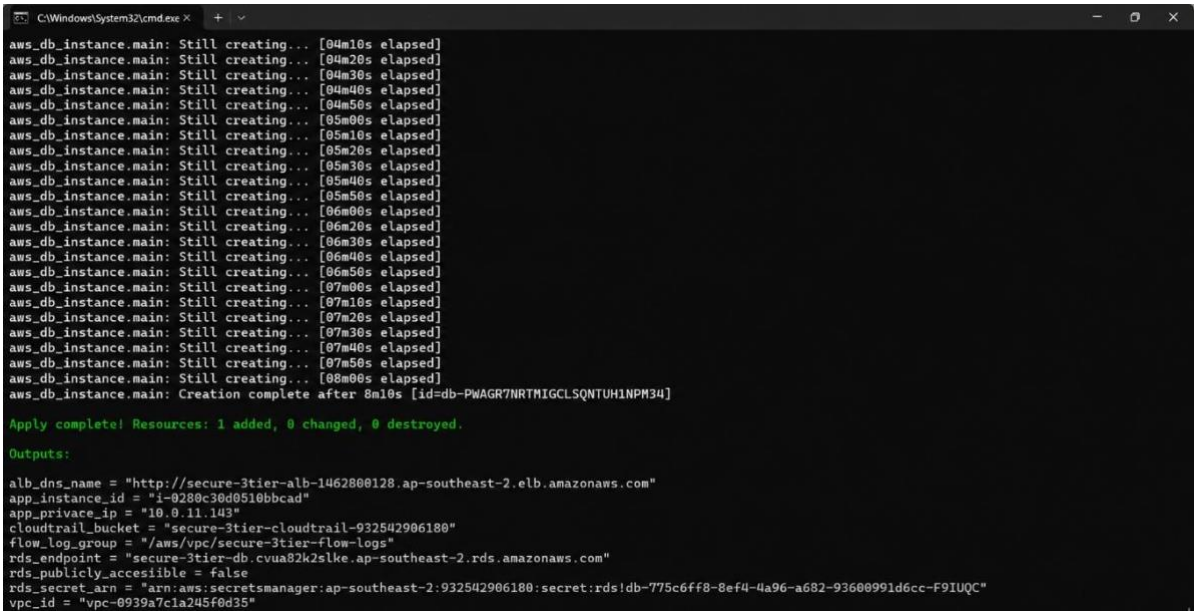


Figure 1. `terraform apply` finished successfully and printed the ALB URL, instance ID, RDS endpoint and log group.



Figure 2. The application page, served from a private subnet, loads through the ALB DNS name.

4.2 Network: VPC, subnets, gateways and routes

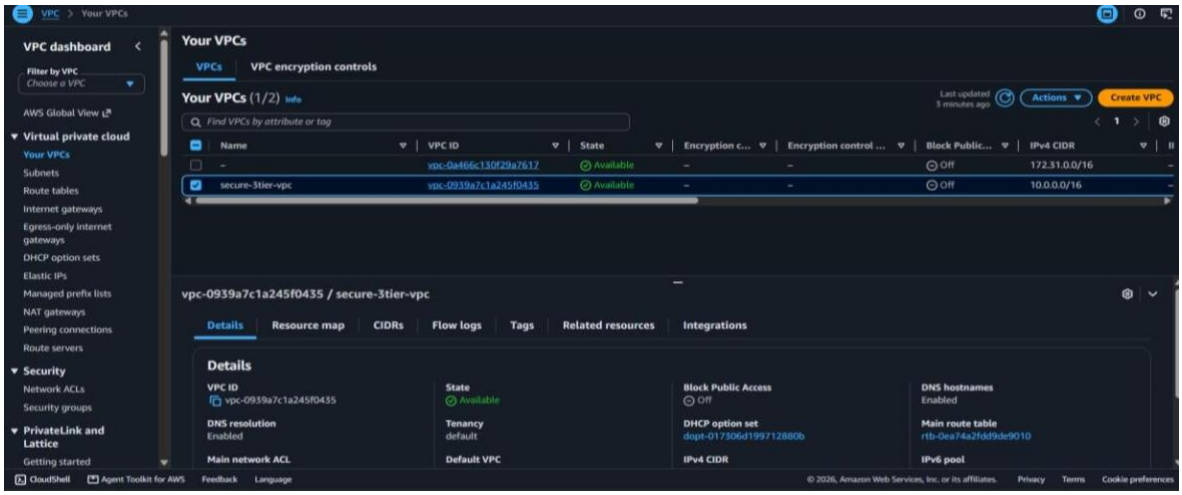


Figure 3. `secure-3tier-vpc` with CIDR 10.0.0.0/16.

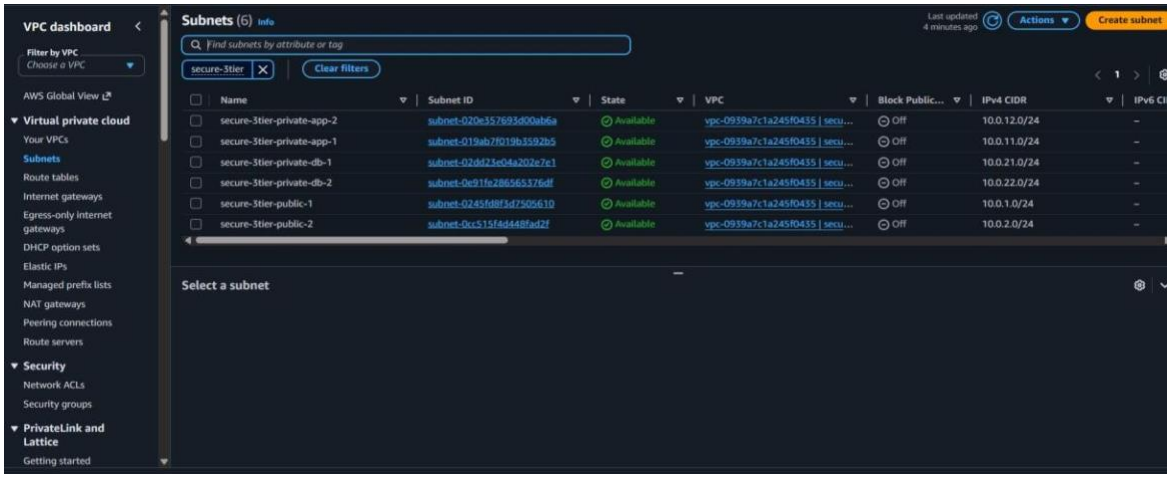


Figure 4. Six subnets across two availability zones: public, private-app and private-db.

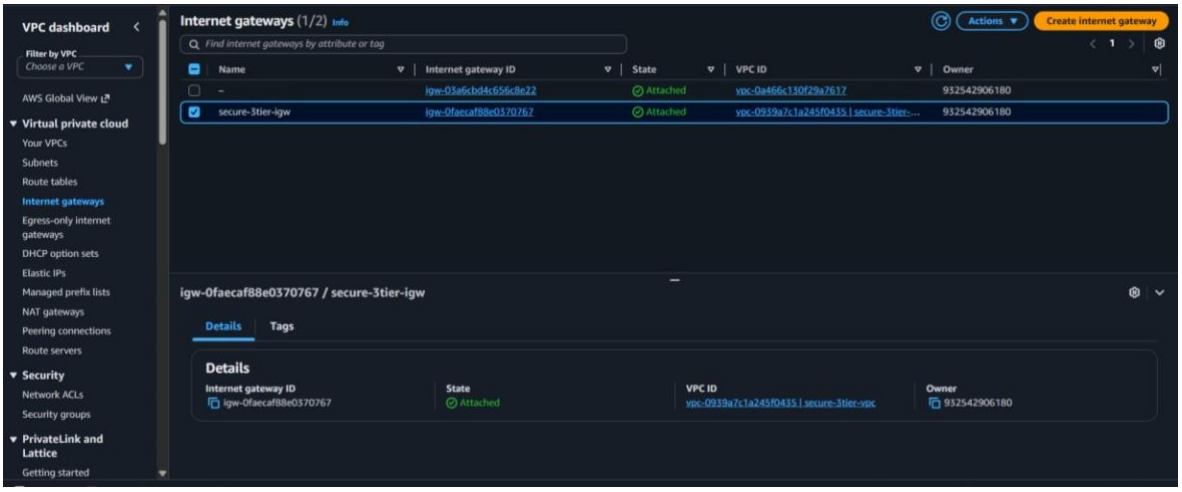


Figure 5. The Internet Gateway is attached to the VPC.

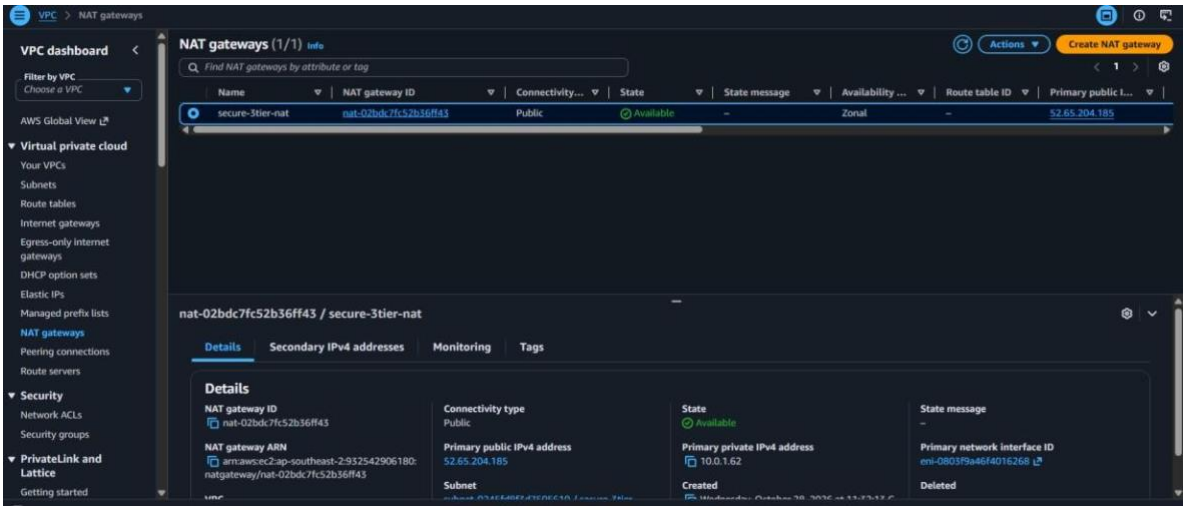


Figure 6. The NAT Gateway is Available in a public subnet with an Elastic IP.

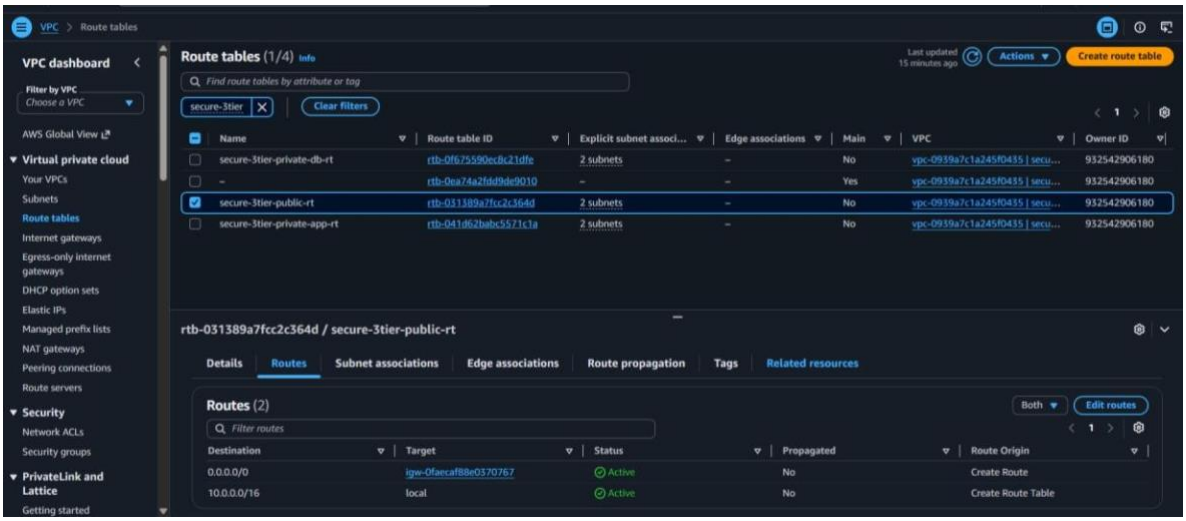


Figure 7. Public route table: `0.0.0.0/0` goes to the Internet Gateway.

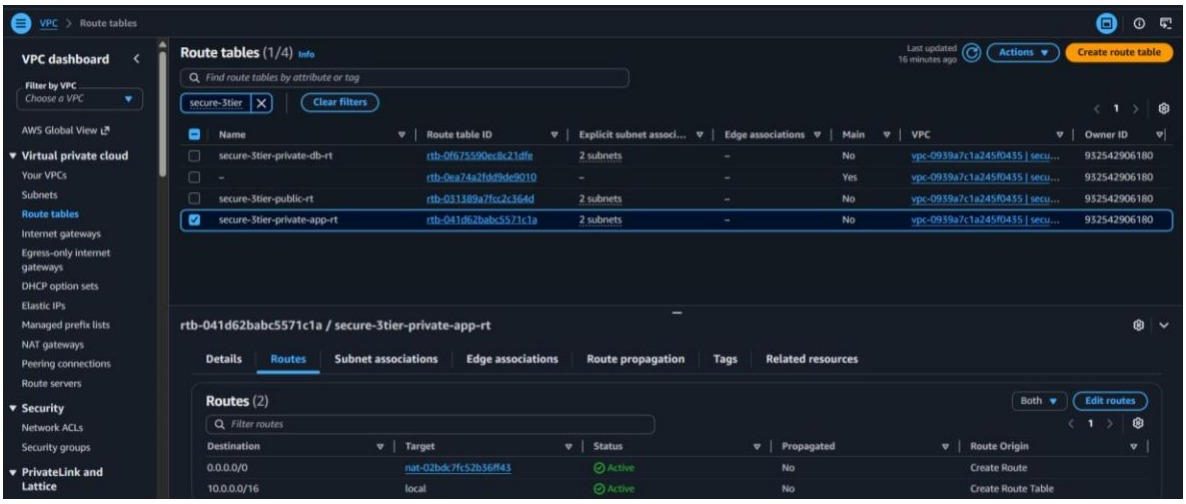


Figure 8. Application route table: `0.0.0.0/0` goes to the NAT Gateway, so the server can make outbound requests only.

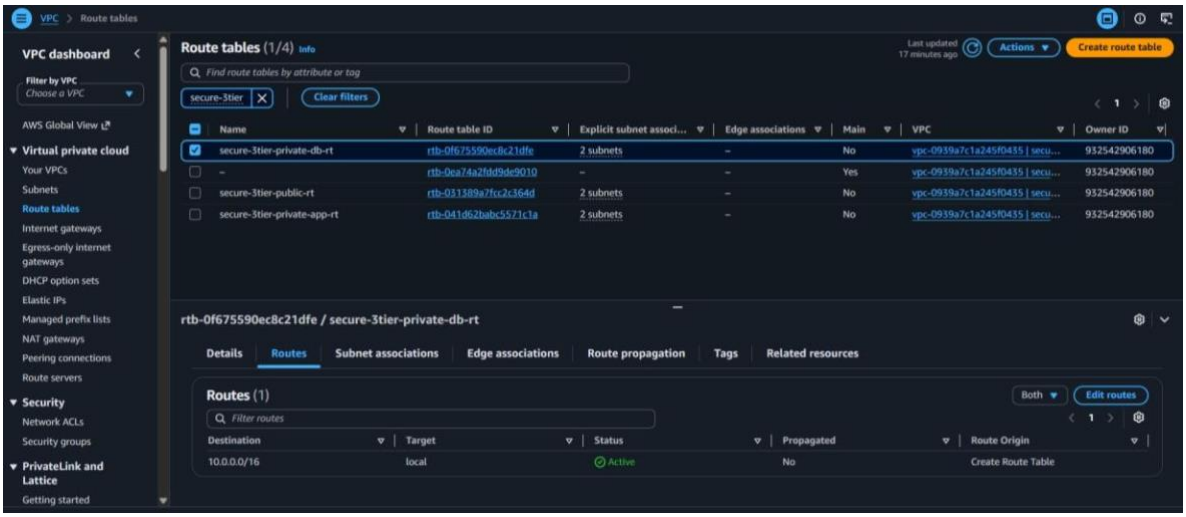


Figure 9. Database route table: only the local VPC route. The database subnets have no path to the internet.

4.3 Load balancer and application server

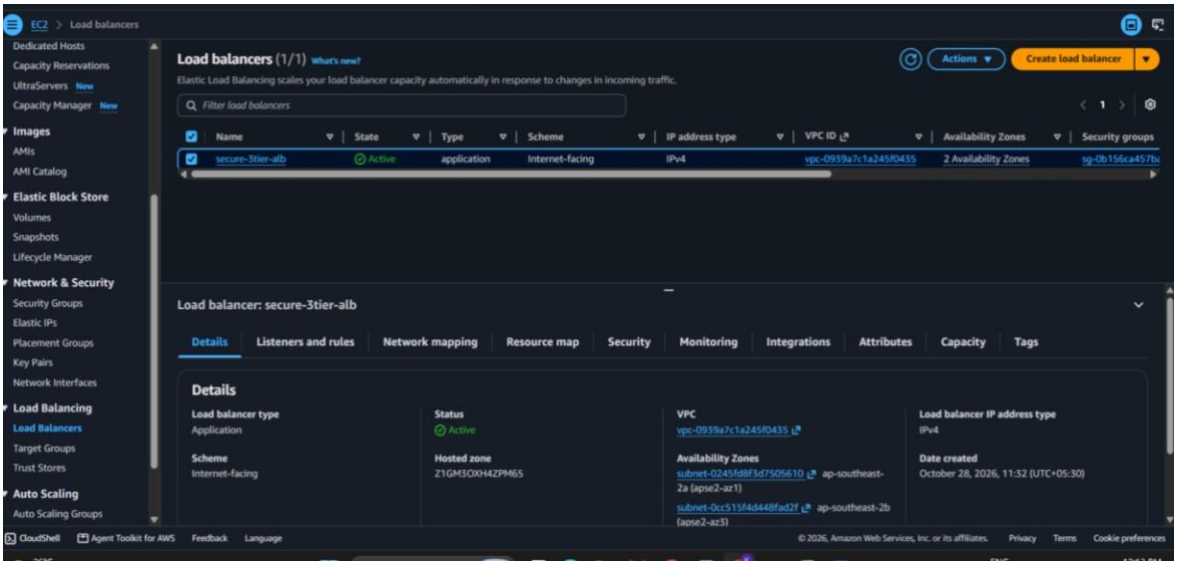


Figure 10. The ALB is internet-facing and Active across two availability zones.

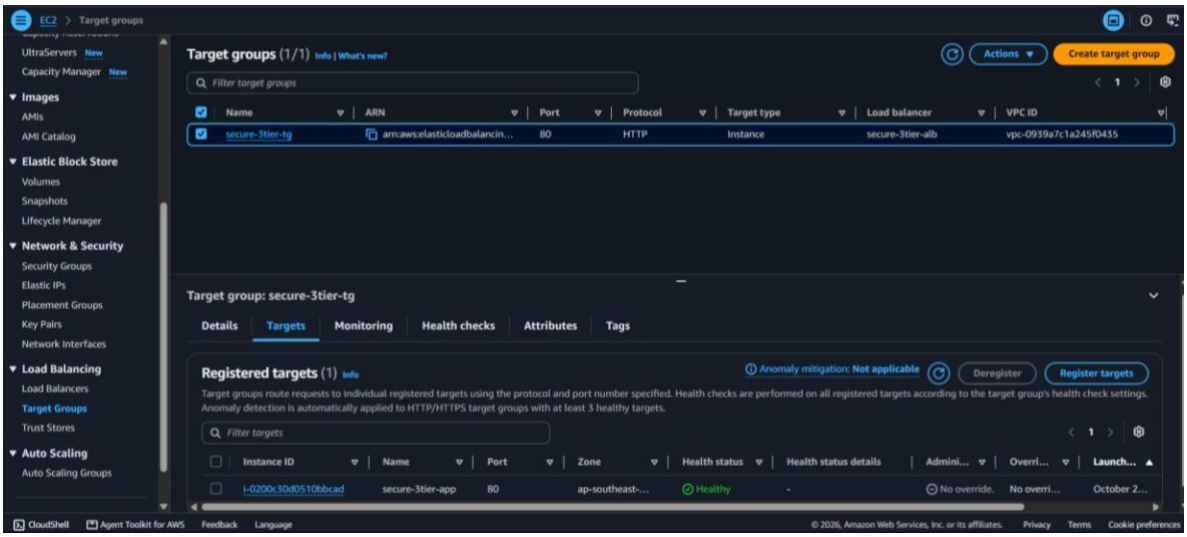


Figure 11. The EC2 instance is registered in the target group and reports Healthy.

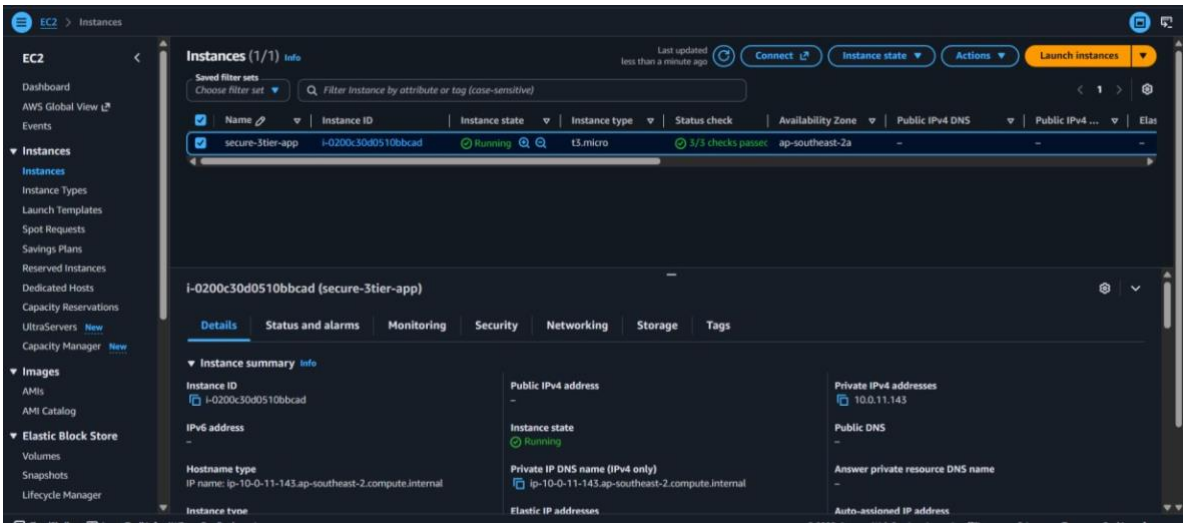


Figure 12. The application server has no public IPv4 address. Its private IP is 10.0.11.143.

4.4 Security groups

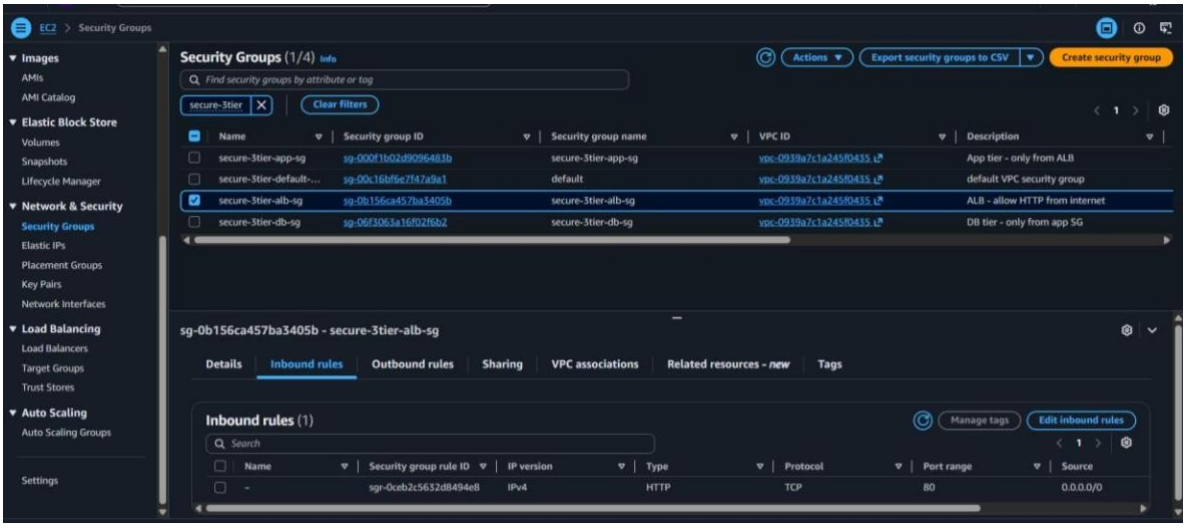


Figure 13. ALB security group: HTTP port 80 from `0.0.0.0/0`.

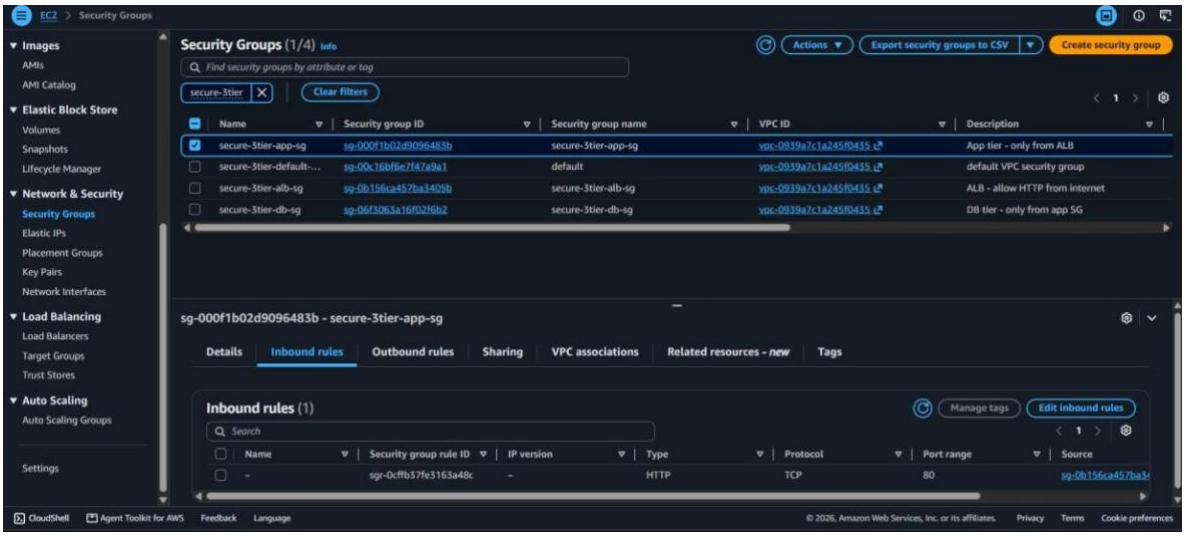


Figure 14. Application security group: port 80 only from the ALB security group.

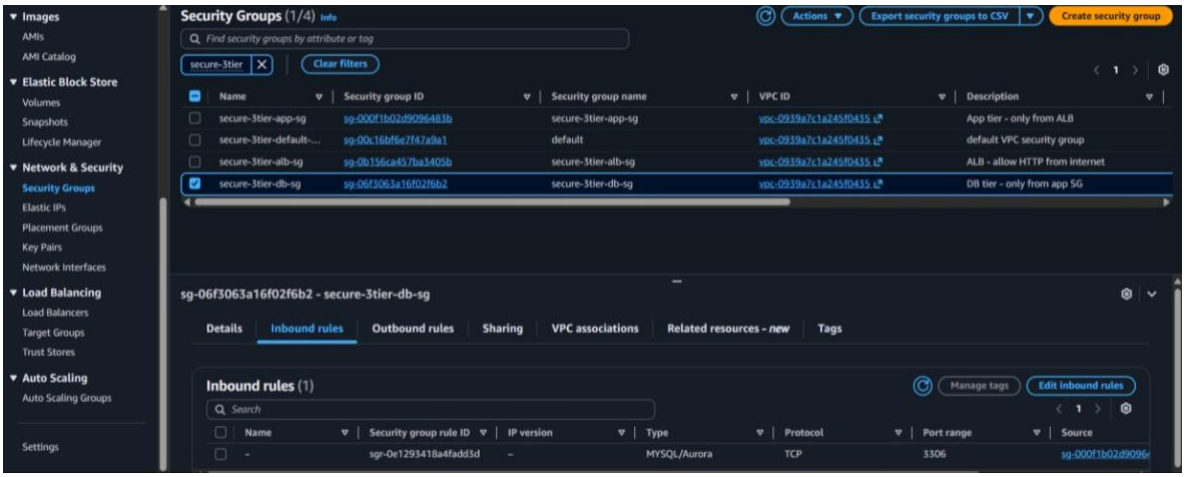


Figure 15. Database security group: MySQL port 3306 only from the application security group.

5. Database Security Evidence

The database is not publicly accessible, is encrypted, and accepts connections only from the application server. This was verified in three ways: configuration, security group rules, and live connection tests from two locations.

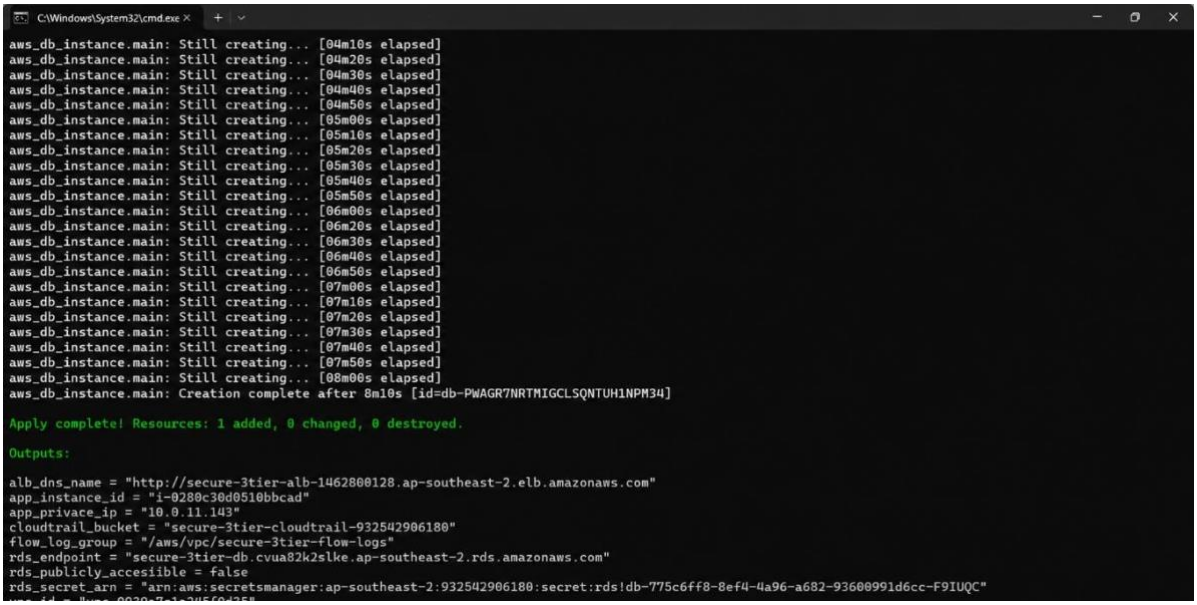


Figure 16. `aws rds describe-db-instances` reports `PubliclyAccessible = False`, `Encrypted = True` and the private DB subnet group.

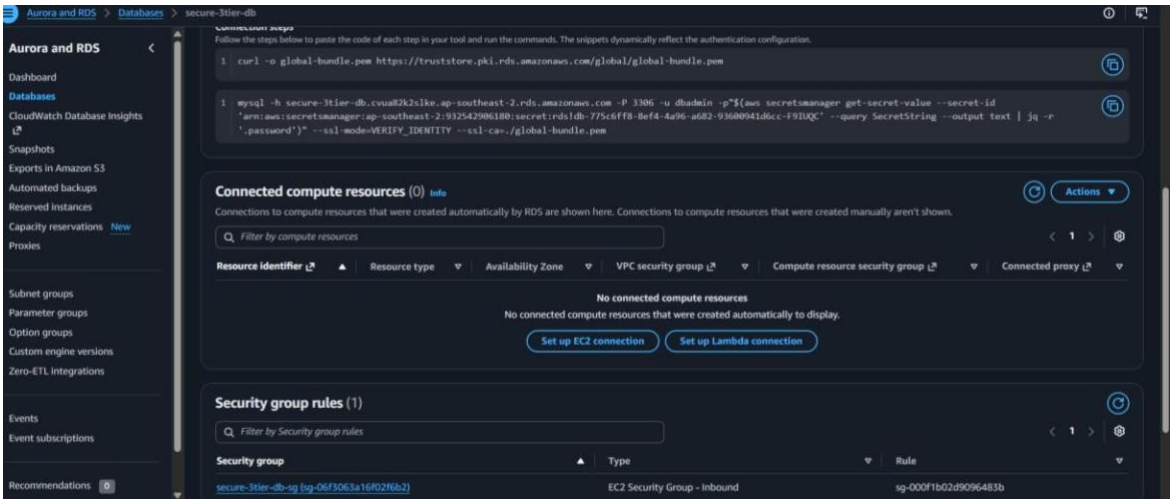


Figure 17. The RDS console lists a single inbound rule on `secure-3tier-db-sg`, sourced from the application security group.

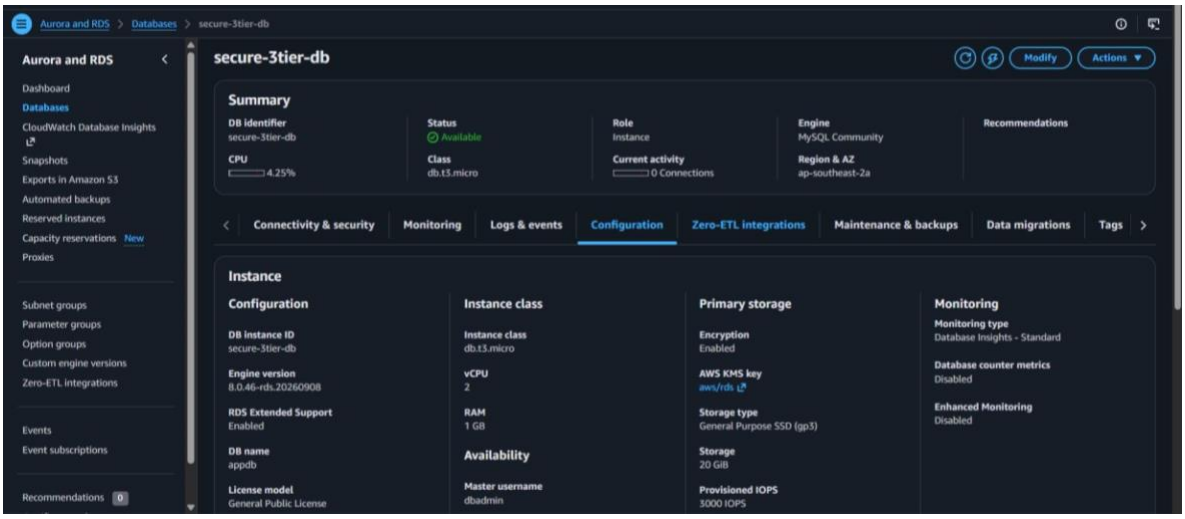


Figure 18. RDS configuration shows storage encryption Enabled with the AWS-managed KMS key (aws/rds), instance class db.t3.micro and 20 GiB of gp3 storage.

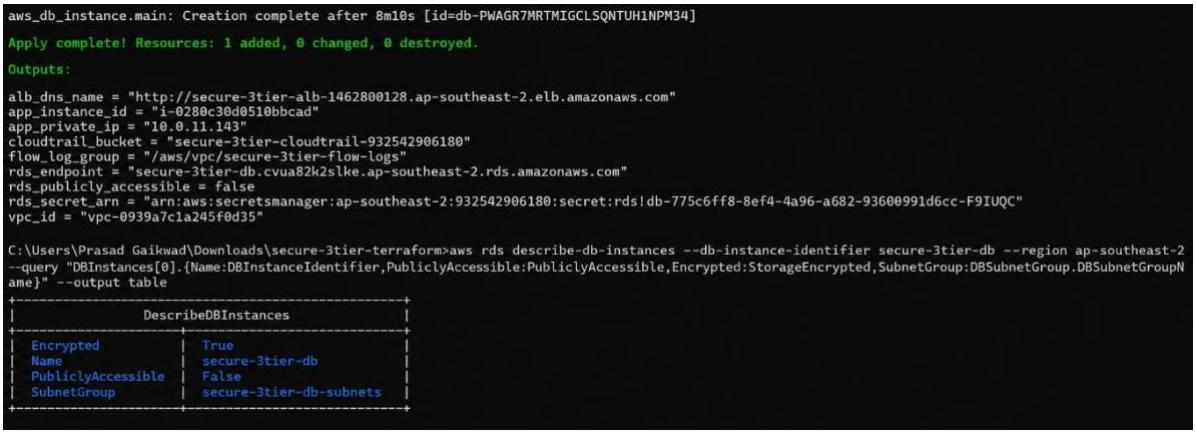


Figure 19. A TCP test from a laptop on the internet to the RDS endpoint on port 3306 fails. The endpoint resolves only to a private address (10.0.21.239) in the database subnet.

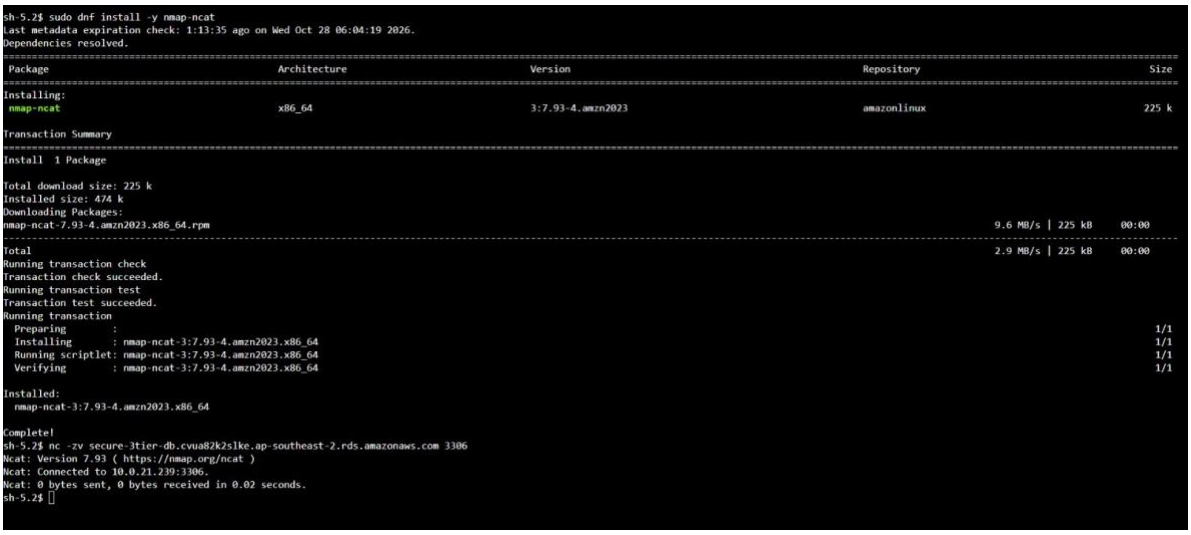


Figure 20. From the application server (reached with Session Manager, no SSH), the same test succeeds: `Connected to 10.0.21.239:3306`.

The two tests together prove the requirement: the database is unreachable from the internet and reachable only from the application tier.

6. Security Monitoring

Two services provide basic security monitoring: VPC Flow Logs record network traffic, and CloudTrail records every API call made in the account.

6.1 VPC Flow Logs

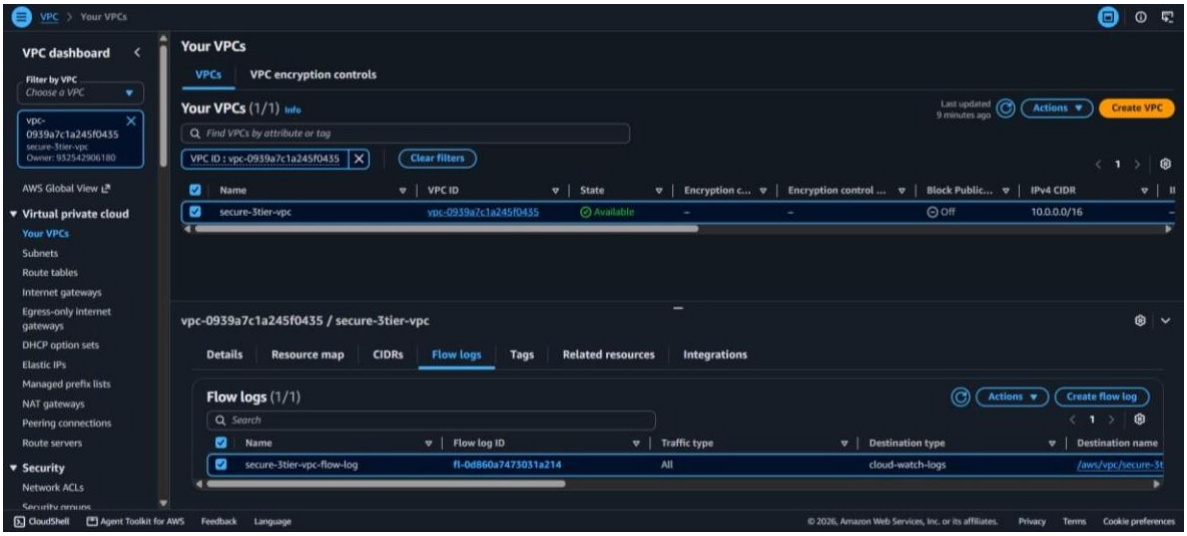


Figure 21. A flow log on the VPC captures ALL traffic and sends it to CloudWatch Logs.

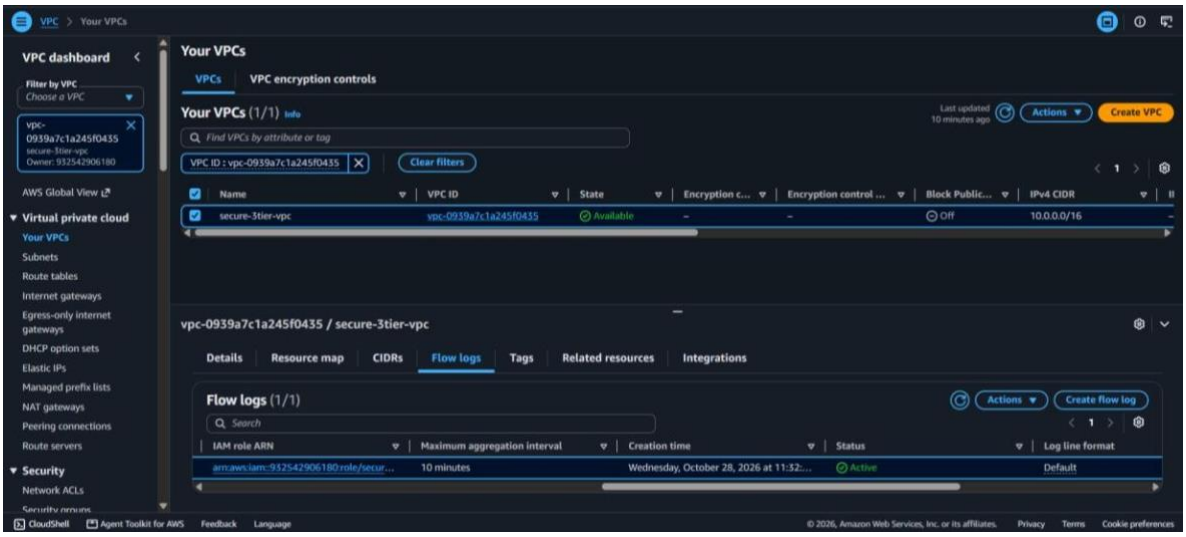


Figure 22. The flow log status is Active with a 10-minute aggregation interval.

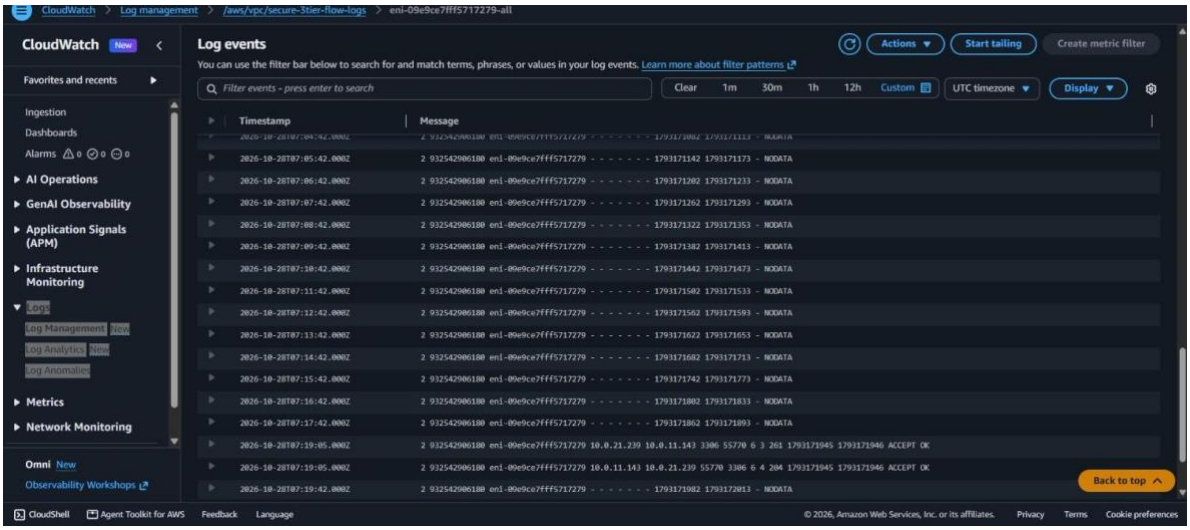


Figure 23. Flow log records in CloudWatch. The ACCEPT lines show MySQL traffic on port 3306 between the application server (10.0.11.143) and the database (10.0.21.239), confirming the only permitted path to the database.

6.2 CloudTrail

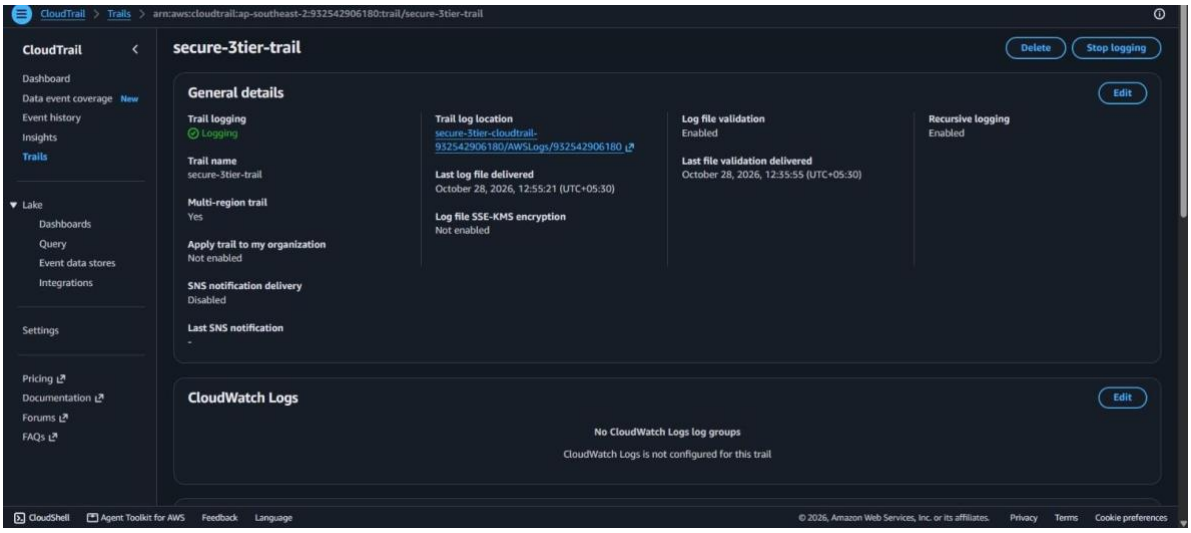


Figure 24. `secure-3tier-trail` is logging, multi-region, with log file validation enabled and logs delivered to the private S3 bucket.

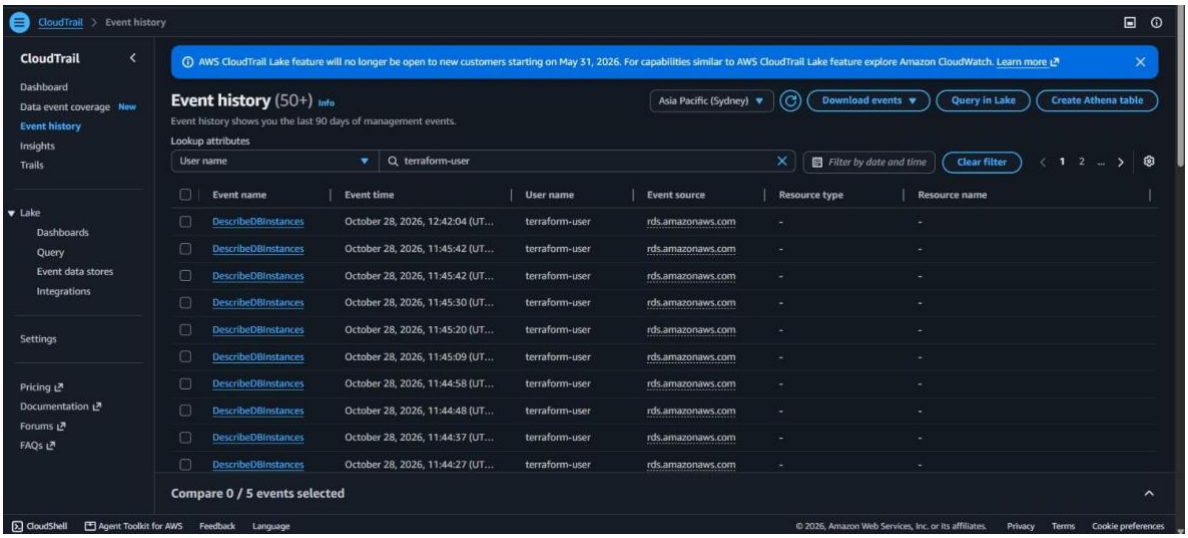


Figure 25. Event history shows API activity by `terraform-user`, giving a full audit trail of who changed what and when.

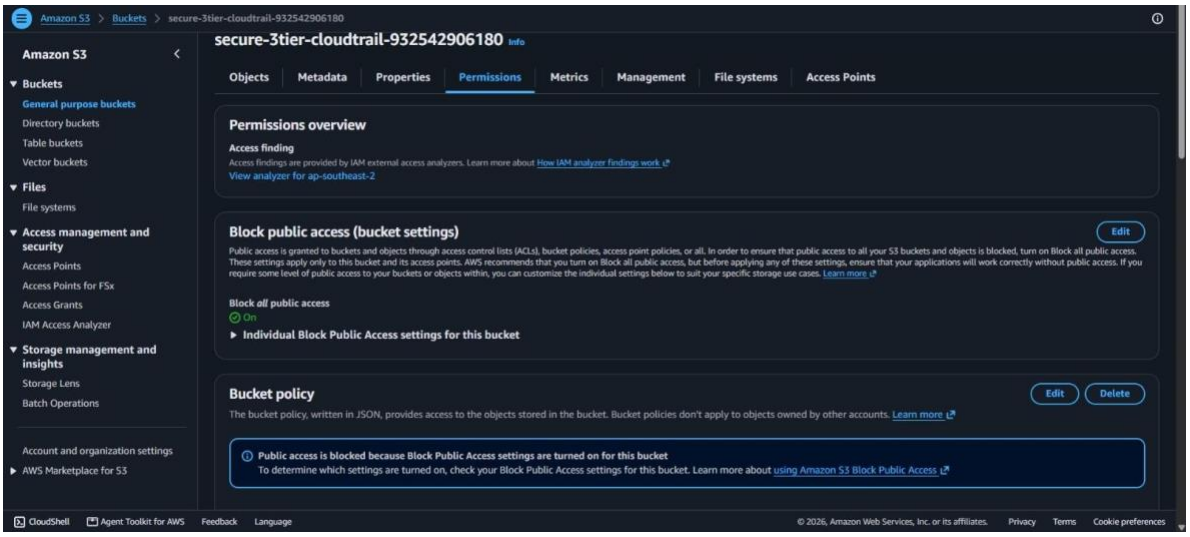


Figure 26. The CloudTrail log bucket has Block all public access turned On, so audit logs cannot be exposed.

7. Security Best Practices Applied

Security is layered: no single control is relied on alone.

Control	How it is implemented	Risk reduced
Network isolation	Only the ALB sits in public subnets; EC2 and RDS have no public IP	Direct internet attacks on servers and database
Isolated database subnets	DB route table has no route to the Internet Gateway or NAT Gateway	Data exfiltration and inbound access
Least-privilege security groups	Internet to ALB, ALB to App, App to DB, each by security group ID	Lateral movement between tiers
No SSH exposure	Admin access through SSM Session Manager, no port 22 rule	Brute-force and key theft
IMDSv2 enforced	`http_tokens = required` on the EC2 instance	Credential theft through SSRF
Encryption at rest	Encrypted EBS volume, encrypted RDS storage, encrypted S3 bucket	Data exposure from disk or snapshot access
Secret handling	RDS master password generated and stored in AWS Secrets Manager	Passwords in code or Terraform variables
Locked default security group	All rules removed from the VPC default group	Accidental exposure of new resources
Audit logging	CloudTrail multi-region trail with log file validation	Undetected changes, tampered logs
Traffic visibility	VPC Flow Logs capture ALL traffic	Blind spots in network activity
Private log storage	CloudTrail S3 bucket blocks public access and denies non-TLS requests	Leaked audit logs

8. Challenges Faced and Resolutions

The AWS account belongs to an AWS Organization with restrictions, which caused three errors during deployment. Each was diagnosed from the Terraform error message and fixed.

Problem	Cause	Resolution
`ssm:GetParameter` denied	A service control policy (SCP) blocked reading the public AMI parameter in the first region tried	Confirmed with the AWS CLI that the Sydney region (ap-southeast-2) allowed the lookup, and moved the deployment there
`ec2:DescribeImages` denied	The same SCP blocked the alternative AMI lookup in the original region	Redeployed in the permitted region using the SSM AMI lookup
`FreeTierRestrictionError` on RDS	The account is on a free plan, which allows a backup retention of only 1 day	Changed `backup_retention_period` from 7 to 1 in `database.tf` and re-applied; only the remaining resources were created

Terraform's state tracking meant the re-apply created only the resources that had failed, without rebuilding the network or the load balancer.

9. Cost, Cleanup and Possible Improvements

The NAT Gateway and the RDS instance are the main running costs, so the environment is removed with `terraform destroy` as soon as the evidence has been collected. The S3 bucket is created with `force_destroy` and RDS with `skip_final_snapshot` so teardown completes without manual steps. These two settings are lab conveniences and should be reversed in production.

The design can be hardened further:

Improvement	Benefit
Auto Scaling Group across both application subnets	True scalability and automatic replacement of failed servers
HTTPS listener with an ACM certificate, HTTP-to-HTTPS redirect	Encryption in transit between users and the ALB
Multi-AZ RDS and one NAT Gateway per availability zone	High availability across zone failures
AWS WAF on the ALB	Protection from common web attacks
GuardDuty and CloudWatch alarms on REJECT spikes	Active detection of threats instead of passive logging
CloudTrail delivery to CloudWatch Logs and SSE-KMS encryption	Faster investigation and customer-managed encryption keys
Remote Terraform state in S3 with locking	Safe collaboration and state protection

10. Conclusion

The project delivered a working three-tier architecture in which the database is provably unreachable from the internet and reachable only from the application server. Evidence: a connection test from a laptop to the RDS endpoint on port 3306 failed, while the same test from the private EC2 instance succeeded, and VPC Flow Logs recorded the accepted application-to-database traffic.

Defining everything in Terraform made the deployment repeatable, reviewable and easy to tear down, and the layered controls (subnet isolation, security group chaining, encryption, Session Manager, Flow Logs and CloudTrail) meet the security objectives of the assignment.